



Aula 11 – Concorrência e Níveis de Isolamento

Controle simultâneo de transações e prevenção de conflitos no banco de dados

Objetivos da Aula



Concorrência

Entender o que é **concorrência** em bancos de dados.



Níveis de Isolamento

Conhecer os **níveis de isolamento**: READ COMMITTED, REPEATABLE READ e SERIALIZABLE.



Bloqueios

Compreender **bloqueios e deadlocks**.



Simulação Prática

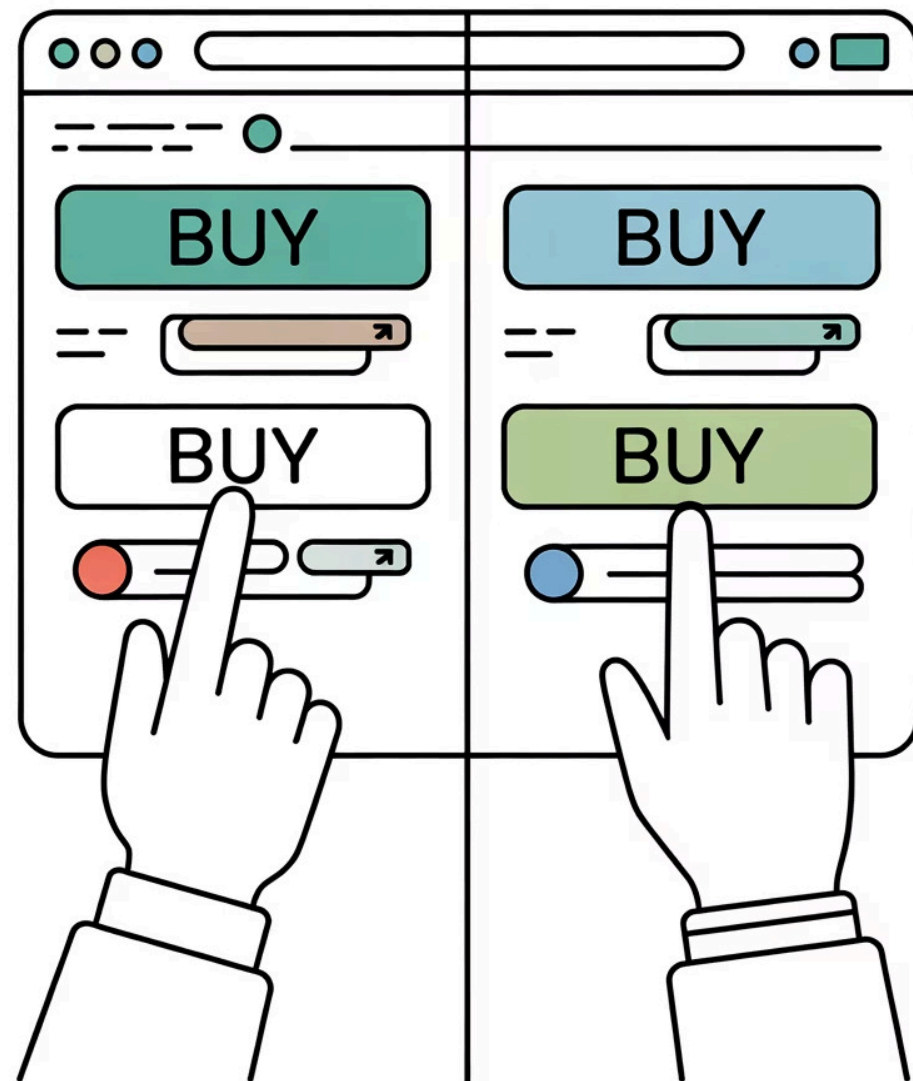
Simular o problema clássico: *vários clientes comprando o mesmo produto ao mesmo tempo*.

Motivação (caso real)

Imagine um site de e-commerce com **apenas 1 unidade** de um produto. Dois clientes clicam em "Comprar" ao mesmo tempo.

💡 O que acontece se o banco não tiver controle de concorrência?

➡ Ambos podem "comprar" e o estoque fica negativo!



Conceitos Fundamentais

O que é Concorrência?

Concorrência ocorre quando **duas ou mais transações** acessam os **mesmos dados ao mesmo tempo**. O banco precisa garantir que o resultado final seja **correto e consistente**, independentemente da ordem de execução.



Problemas comuns de concorrência

Problema	Descrição	Exemplo
Dirty Read	Ler dados que ainda não foram confirmados (sem COMMIT).	Ver estoque alterado antes da transação ser confirmada.
Non-Repeatable Read	O mesmo SELECT retorna resultados diferentes na mesma transação.	Consultar o estoque duas vezes e ver valores diferentes.
Phantom Read	Linhas "fantasmas" aparecem ou somem durante a transação.	Um cliente vê produtos disponíveis que outro acabou de comprar.

Propriedades ACID e Concorrência

Relembrando ACID

A concorrência está diretamente ligada ao "I" — **Isolamento**.

Cada transação deve funcionar **como se fosse a única executando no banco**.

Isolamento em níveis

Nem sempre é possível ter isolamento total sem perda de desempenho. Por isso, os bancos implementam **níveis de isolamento** que equilibram **segurança x performance**.



Níveis de Isolamento (PostgreSQL)

1 READ COMMITTED (Leitura Confirmada)

- Cada instrução SQL dentro da transação **vê apenas dados já confirmados (COMMIT)**.
- Se outro usuário confirmar mudanças, a próxima query já vê o novo valor.

💡 Exemplo: **Transação A** lê estoque = 10. **Transação B** faz COMMIT com estoque = 9. Se **A** consultar de novo, verá **9**.

Níveis de Isolamento (PostgreSQL)

1 READ COMMITTED (Leitura Confirmada)

É o nível de isolamento padrão do PostgreSQL. Garante que cada instrução SQL vê apenas dados que já foram **confirmados (COMMIT)** por outras transações.

💡 **Exemplo:** Cliente A vê estoque = 10. Cliente B compra e dá COMMIT (estoque = 9). Se Cliente A consultar de novo, verá **9**.

✓ Vantagens

- Rápido e eficiente, ideal para a maioria das aplicações.
- Evita leituras sujas (Dirty Reads).
- Permite boa concorrência.

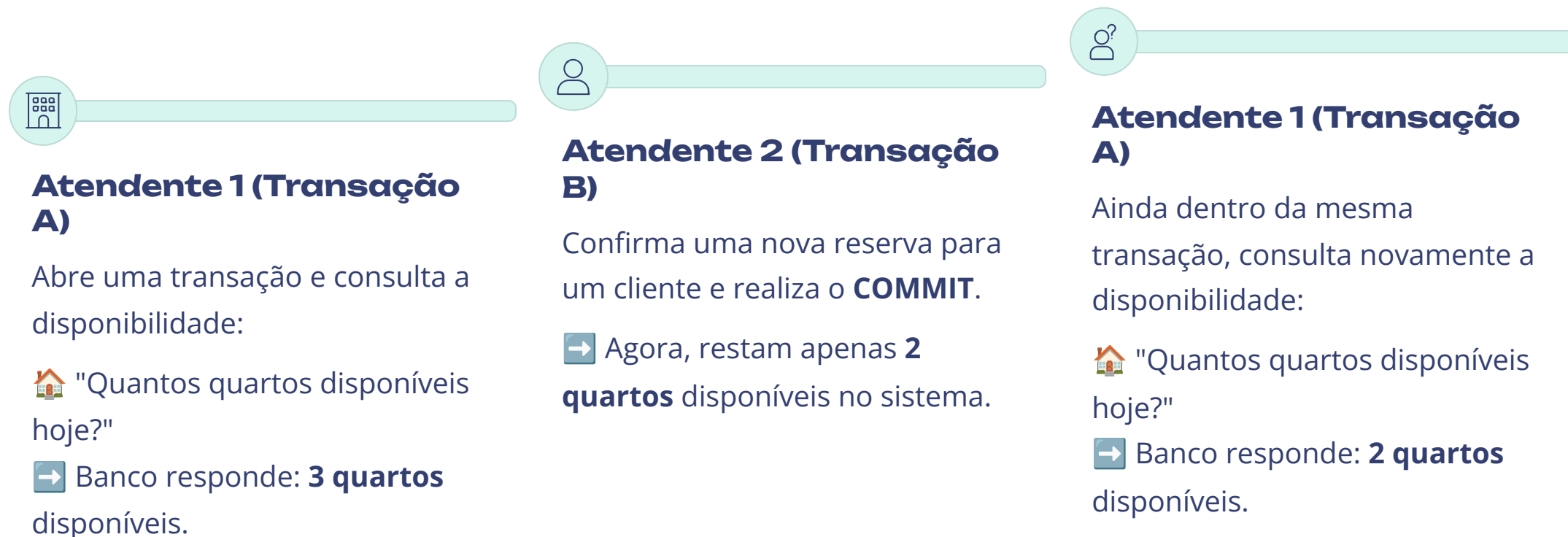
⚠ Desvantagens

- Pode ocorrer leitura não repetível (dados mudam na mesma transação).
- Pode gerar leituras fantasmas (novos registros aparecem/somem).
- Menor consistência para operações muito críticas.

Em resumo: no READ COMMITTED, cada leitura pega a versão mais recente e confirmada dos dados — rápido e seguro o suficiente para a maioria dos sistemas do dia a dia.

Exemplo prático – Limites do READ COMMITTED

Vamos simular um sistema de reservas de quartos de pousada para entender as implicações do **READ COMMITTED**:



💡 O que observamos neste cenário:

- ❌ **Leitura não repetível:** A mesma consulta (SELECT) retornou valores diferentes dentro da mesma transação.
- 👻 **Leitura fantasma:** Se um novo quarto fosse adicionado por outra transação e confirmado, ele "apareceria" do nada na segunda consulta da Transação A.
- ⚠️ **Menor consistência:** A contagem de quartos mudou enquanto a Transação A ainda estava operando, o que pode levar a decisões incorretas.

📝 No **READ COMMITTED**, o banco sempre mostra a versão mais atual — o que é ótimo para desempenho, mas pode ser problemático se você precisa que os dados permaneçam estáveis durante toda a transação.

2 REPEATABLE READ - Leitura Repetível

- Todos os dados lidos pela transação **permanecem consistentes até o fim**.
- Mesmo que outros usuários façam COMMIT, **a transação não vê as mudanças**.

  Exemplo: A consulta sempre verá estoque = 10 até o COMMIT, mesmo que outro cliente já tenha comprado.

2 REPEATABLE READ - Leitura Repetível

É um nível de isolamento intermediário que garante que todos os dados lidos dentro de uma transação permanecem iguais até o fim dela, mesmo que outros usuários façam alterações e deem COMMIT.

💡 Em outras palavras: o banco “congela” a visão dos dados no momento da primeira leitura.

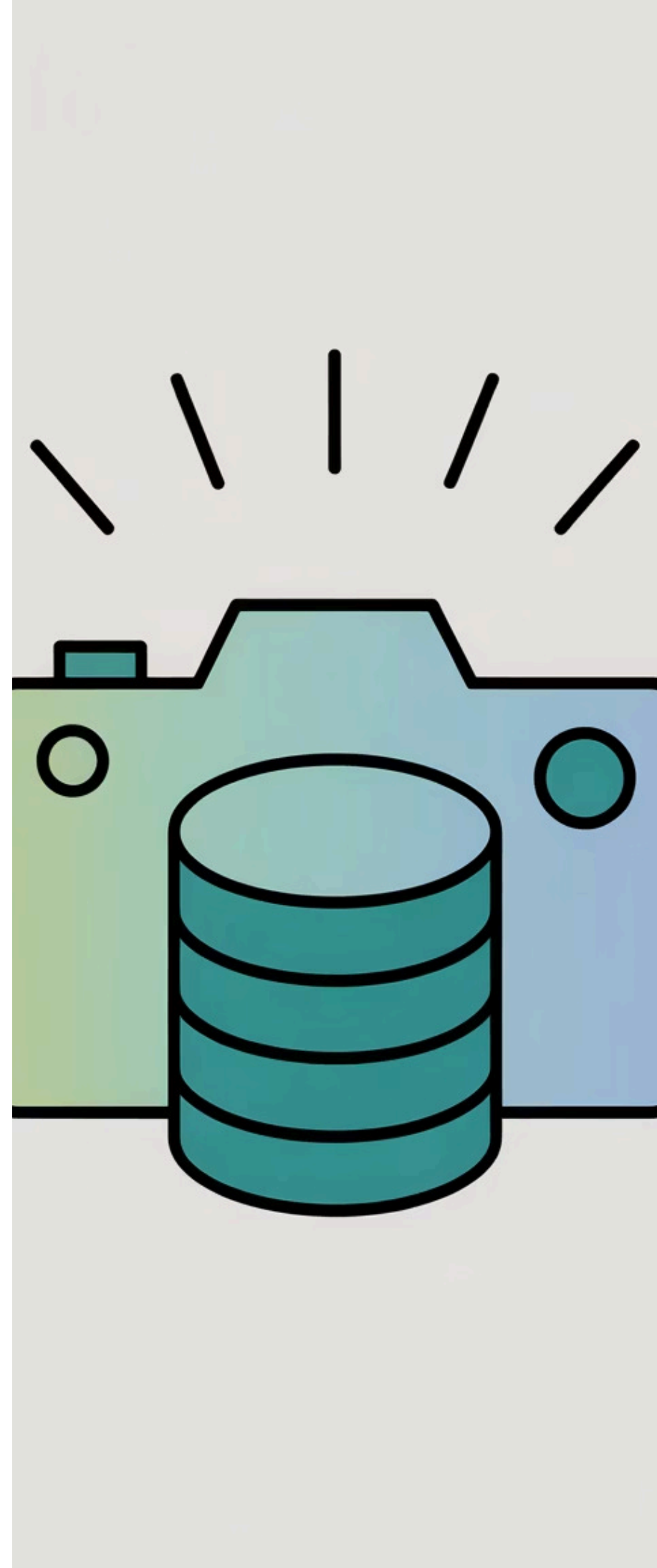
✓ Vantagens

- Evita leituras inconsistentes (non-repeatable read).
- Garante estabilidade nos resultados dentro da mesma transação.
- Ideal para relatórios e cálculos que dependem de consistência.

⚠ Desvantagens

- Pode gerar “phantom reads” (novos registros inseridos ainda aparecem).
- É mais lento que o modo padrão (READ COMMITTED).

"O REPEATABLE READ é como tirar uma foto dos dados no momento da leitura — enquanto a transação não termina, o banco garante que tudo continue igual àquela foto."



Phantom Read – Leitura Fantasma

Ocorre quando uma transação executa consultas idênticas, mas novas linhas aparecem ou desaparecem porque outra transação inseriu ou excluiu registros no mesmo intervalo de busca.

💡 É como se novos dados **“surgissem do nada”** enquanto a transação ainda estava aberta.

Exemplo Prático (Varejo/E-commerce):

01

Gerente Relatório

Gerente executa relatório listando pedidos.

02

Novo Pedido

Outro funcionário insere um novo pedido.

03

Re-execução

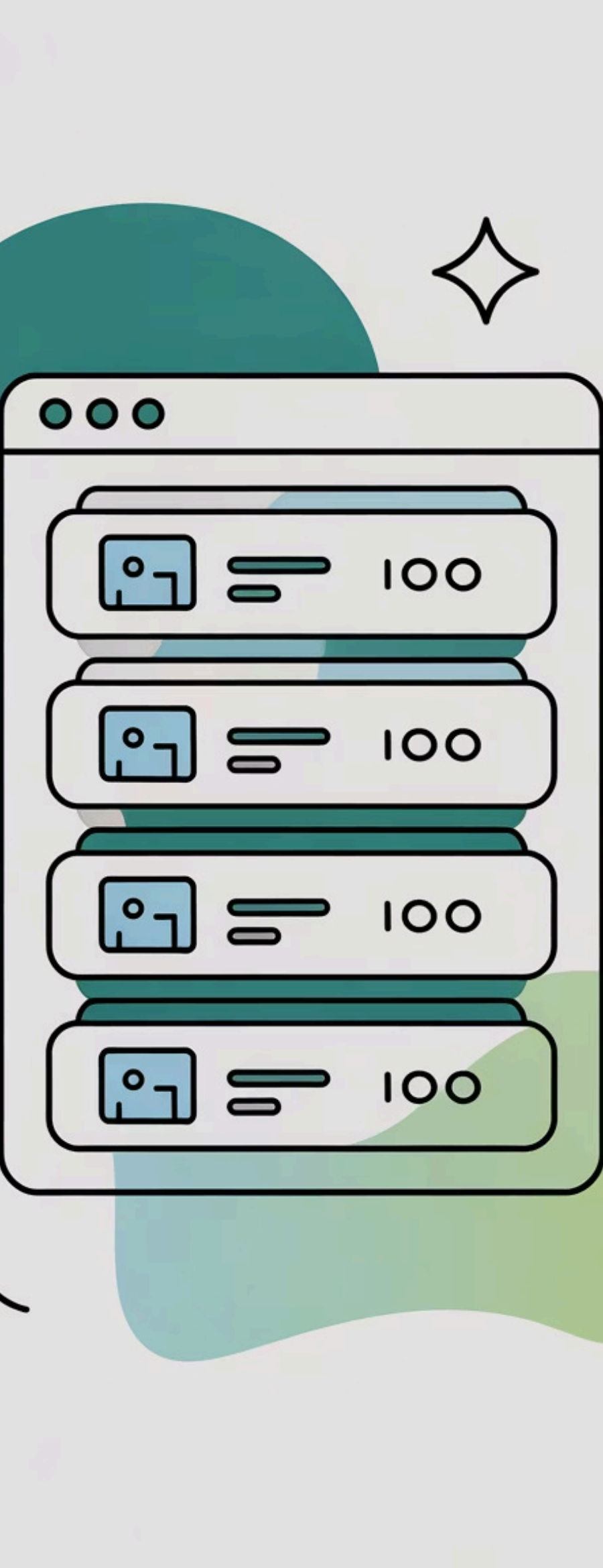
Gerente executa o mesmo relatório e o novo pedido aparece.

➔ Isso é um Phantom Read.

Acontece porque o **REPEATABLE READ** mantém a consistência das linhas já lidas, mas não impede a inserção de novas linhas que atendem ao mesmo filtro.

Para evitar, use o nível de isolamento **SERIALIZABLE**, que bloqueia a criação de novos registros conflitantes durante a transação.

"O Phantom Read é como abrir uma lista de produtos, piscar os olhos e ver novos itens que não estavam lá antes — o dado é real, mas surgiu no meio da sua transação."



3 SERIALIZABLE

- É o nível mais seguro e o mais lento.
- O banco **simula a execução das transações em série**, garantindo o mesmo resultado que se fossem executadas uma de cada vez.

💡 Exemplo: Se dois clientes tentarem comprar a última unidade, o banco deixará **apenas um COMMIT**, e o outro **falhará** (erro de serialização).

3 **SERIALIZABLE** – Isolamento Total

É o nível mais alto de isolamento no PostgreSQL. Ele garante que as transações se comportem como se fossem executadas uma de cada vez, sequencialmente, mesmo que ocorram simultaneamente.

💡 O banco simula uma fila de espera, processando cada transação isoladamente para garantir máxima consistência.

Imagine dois clientes tentando comprar a última unidade de um produto. Com **SERIALIZABLE**, apenas uma compra é confirmada. A segunda transação receberá um erro de "serialization failure" e precisará ser refeita.

✓ **Vantagens**

- Máxima integridade e consistência dos dados.
- Evita todos os problemas de concorrência: Dirty Read, Non-Repeatable Read, e Phantom Read.
- Ideal para transações financeiras, controle de estoque e pagamentos.

⚠ **Desvantagens**

- Menor desempenho devido a bloqueios e reprocessamentos frequentes.
- Maior chance de erros de serialização em ambientes com alto volume.
- Pode exigir tratamento de exceções para reexecução de transações.

"O SERIALIZABLE é o modo 'máxima segurança' — ninguém lê nem grava nada que possa interferir na sua transação. É o mais confiável, mas também o mais custoso em desempenho."

Quando usar cada nível

Relatórios e consultas simples

READ COMMITTED

Processos críticos de atualização

REPEATABLE READ

Transações financeiras ou vendas exclusivas

SERIALIZABLE

Quando usar cada nível de isolamento

A escolha do nível de isolamento adequado é crucial para balancear consistência dos dados e desempenho da aplicação. Veja um guia prático para cada nível:

READ COMMITTED (Padrão)	Quando a prioridade é desempenho e pequenas variações de leitura não causam problema.	 Consultas simples, relatórios diários ou dashboards de BI que podem mostrar dados recentes, mesmo que mudem em segundos.
REPEATABLE READ	Quando é importante que os dados não mudem durante toda a transação, mantendo a consistência dos resultados.	 Atualização de estoque ou cálculo de comissão, onde o valor precisa permanecer fixo até o fim da operação.
SERIALIZABLE	Quando nenhum erro é aceitável e os dados precisam ser 100% consistentes, mesmo com alta concorrência.	 Processamento de pagamento, fechamento de caixa ou venda de um item único (último produto em estoque).

 **Lembre-se:** quanto maior o nível de isolamento, maior a garantia de consistência dos dados, mas também menor o desempenho.

O que é o Nível de Isolamento da Transação?

💡 Cada transação no PostgreSQL pode ser executada com um nível de isolamento diferente, controlando como ela enxerga as alterações feitas por outras transações concorrentes.

Os níveis de isolamento definem o grau em que as operações de uma transação são protegidas de outras transações simultâneas. Eles determinam quais problemas de concorrência (como Dirty Read, Non-Repeatable Read e Phantom Read) podem ser evitados, impactando diretamente a consistência dos dados e o desempenho do sistema.

A configuração do nível de isolamento pode ser feita no início de cada transação específica, ou globalmente para todas as transações dentro da sessão atual:

```
SET TRANSACTION ISOLATION LEVEL <NÍVEL>;
```

Ou, para definir o comportamento padrão para transações futuras na sessão:

```
SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL <NÍVEL>;
```

2 REPEATABLE READ – Leitura Repetível

Neste nível, uma vez que uma transação lê dados, ela garante que esses dados não serão alterados por outras transações até que a transação atual seja concluída. É como se você tirasse uma "foto" dos dados no início da transação.

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
SELECT * FROM produto WHERE id = 123; -- "foto" dos dados neste momento  
  
-- Se outra transação tentar alterar 'produto.id = 123',  
-- ela será bloqueada ou a sua transação verá o valor original.  
  
COMMIT;
```

💬 Este nível evita leituras sujas (Dirty Read) e leituras não repetíveis (Non-Repeatable Read), mas ainda permite que novos registros (phantoms) apareçam se outras transações os inserirem no intervalo de busca.

"O REPEATABLE READ congela a visão dos dados para a transação, assegurando que o que foi lido uma vez permaneça o mesmo, independentemente das alterações de outras transações. Ideal para cenários que exigem consistência de leitura."

3 **SERIALIZABLE** – Simulação e Conflito

O nível **SERIALIZABLE** força as transações a agirem como se fossem executadas uma após a outra, mesmo quando operam concorrentemente. Isso garante a máxima consistência dos dados, simulando uma execução em série.

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  
-- Exemplo: Atualização de estoque para um produto  
UPDATE produto SET estoque = estoque - 1 WHERE id_produto = 1;  
  
COMMIT;
```

💡 **Comportamento em Conflito:** Se duas transações tentarem atualizar o mesmo registro que gera um conflito de serialização, o PostgreSQL detectará a inconsistência e abortará uma delas com um erro de `serialization_failure`. A transação abortada precisará ser reexecutada.



Como testar em sala

1. Use **duas janelas do pgAdmin**.
2. Configure **diferentes níveis de isolamento** nas duas conexões.
3. Execute operações simultâneas no mesmo produto.
4. Observe como cada nível reage (quem lê o valor antigo, quem falha, quem espera).

Exemplo – Procedure com Transação **SERIALIZABLE**

1 Criando a tabela base

```
CREATE TABLE produto (  
  id_produto SERIAL PRIMARY KEY,  
  nome VARCHAR(50),  
  estoque INT NOT NULL  
);
```

```
INSERT INTO produto (nome, estoque) VALUES ('Notebook Gamer', 1);
```

2 Criando a Procedure

A procedure **processa uma compra** e atualiza o estoque se houver unidades disponíveis.

```
CREATE OR REPLACE PROCEDURE realizar_compra(p_id_produto INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_estoque INT;
BEGIN
    -- Verifica o estoque com bloqueio (garante exclusividade)
    SELECT estoque INTO v_estoque
    FROM produto
    WHERE id_produto = p_id_produto
    FOR UPDATE;

    IF v_estoque > 0 THEN
        UPDATE produto
        SET estoque = estoque - 1
        WHERE id_produto = p_id_produto;
        RAISE NOTICE 'Compra realizada com sucesso!';
    ELSE
        RAISE NOTICE 'Produto esgotado!';
    END IF;
END;
$$;
```

3 Executando a Procedure com nível SERIALIZABLE

Agora vem a parte prática: o **nível de isolamento é definido antes da chamada**, e não dentro da procedure.

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  
CALL realizar_compra(1);  
  
COMMIT;
```

4 O que acontece ao simular duas compras simultâneas

💡 Abra **duas janelas (sessões)** do PostgreSQL e execute o mesmo código em ambas:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  
CALL realizar_compra(1);  
  
COMMIT;
```

Resultados esperados:

- A **primeira sessão** executa e confirma a compra normalmente.
- A **segunda sessão** tentará fazer o mesmo, mas o PostgreSQL detectará conflito de serialização e retornará:

```
ERROR: could not serialize access due to concurrent update
```

💬 *Isso demonstra que o banco impediu duas transações de alterarem o mesmo registro ao mesmo tempo.*

5 Testando com outros níveis

Podem apenas **mudar o comando SET** e repetir a simulação:

READ COMMITTED:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
CALL realizar_compra(1);  
COMMIT;
```

→ Pode permitir leituras mais rápidas, mas risco de inconsistência se mal controlado.

REPEATABLE READ:

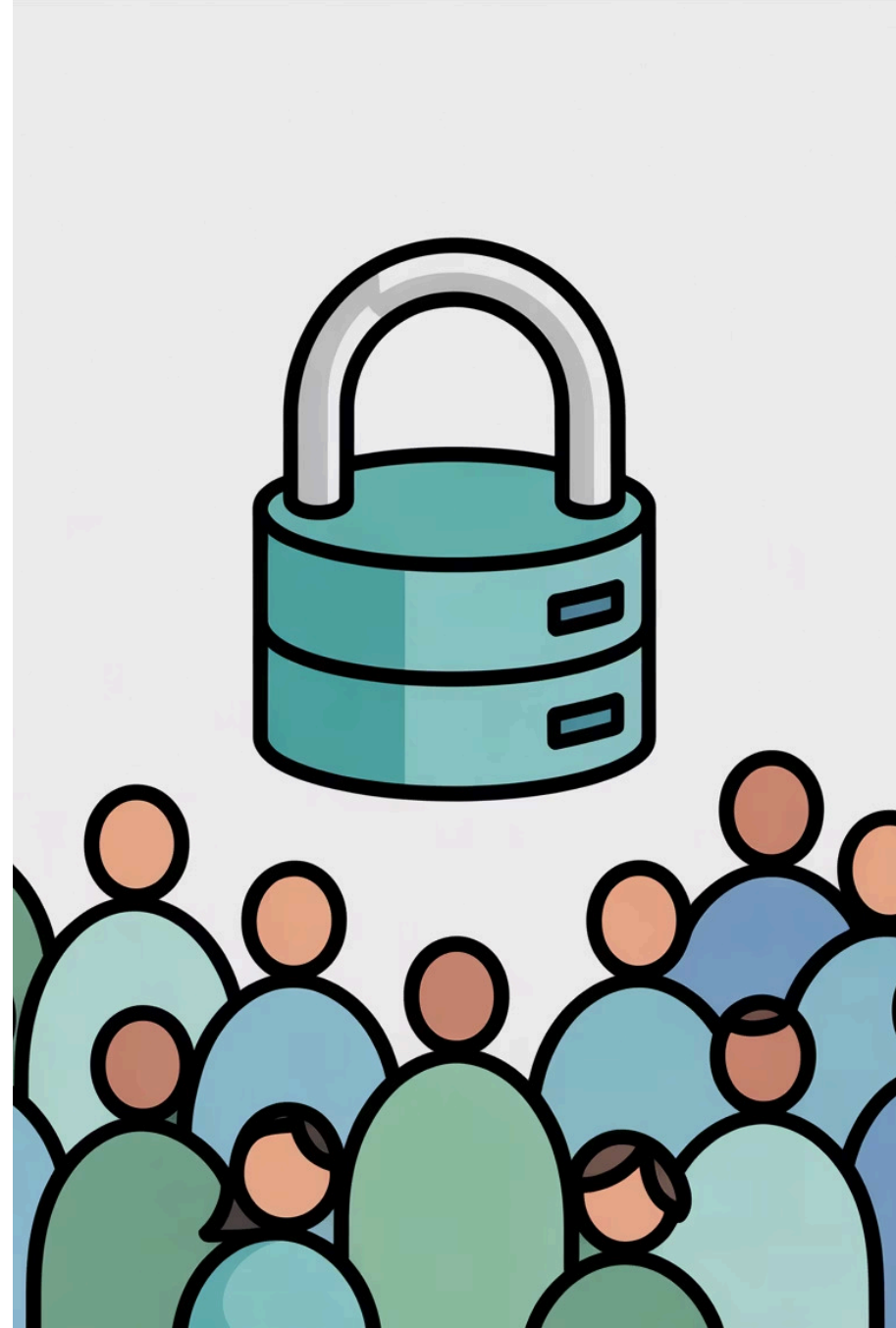
```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
CALL realizar_compra(1);  
COMMIT;
```

→ Garante consistência dentro da transação, mas ainda pode gerar *phantom reads*.

Bloqueios (Locks)

O que são bloqueios?

São mecanismos que o banco usa para **impedir acessos simultâneos conflitantes**. Quando uma transação modifica um registro, ele é **bloqueado até o COMMIT/ROLLBACK**.



Tipos de bloqueios

Tipo	Descrição	Exemplo
Row Lock (Linha)	Bloqueia apenas o registro sendo alterado.	Atualizar estoque de 1 produto.
Table Lock (Tabela)	Bloqueia toda a tabela.	Manutenção ou migração de dados.
Advisory Lock	Bloqueio manual controlado por código.	Uso em sistemas personalizados.

Deadlock

Ocorre quando duas transações ficam **esperando uma à outra liberar o mesmo recurso**.

💡 Exemplo:

- Transação A bloqueia o produto X.
- Transação B bloqueia o produto Y.
- A tenta acessar Y e B tenta acessar X.

➡ Nenhuma prossegue → *deadlock detected*.



Controle de Concorrência no Varejo

Exemplo prático: compra simultânea

Imagine que há **apenas 1 celular no estoque**. Dois clientes tentam comprar **ao mesmo tempo**.

Sem controle, o sistema pode vender **2 produtos com estoque = 0**.

Estratégia de controle

O1

Iniciar uma transação

O2

Verificar o estoque do produto

O3

Se houver unidade disponível → atualizar

O4

Caso contrário → abortar a transação

Simulação prática no PostgreSQL

Estrutura base

```
CREATE TABLE produto (  
  id_produto SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  estoque INT NOT NULL  
);
```

```
INSERT INTO produto (nome, estoque)  
VALUES ('Notebook X', 1);
```

Procedure de compra com controle de concorrência

```
CREATE OR REPLACE PROCEDURE comprar_produto(p_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_estoque INT;
BEGIN
    -- Bloqueia a linha durante a leitura
    SELECT estoque INTO v_estoque
    FROM produto
    WHERE id_produto = p_id
    FOR UPDATE;

    IF v_estoque > 0 THEN
        UPDATE produto
        SET estoque = estoque - 1
        WHERE id_produto = p_id;
        COMMIT;
        RAISE NOTICE 'Compra realizada com sucesso.';
    ELSE
        ROLLBACK;
        RAISE NOTICE 'Produto esgotado. Compra cancelada.';
    END IF;
END;
$$;
```

Explicando o FOR UPDATE

Garante que o registro seja **bloqueado** até o final da transação.

Impede que outra transação leia ou modifique o mesmo produto.

É a base do controle de concorrência em nível de linha.

Simulação em Duas Sessões

Como testar



Passo 1

Abra **duas conexões diferentes** no PostgreSQL (ou duas janelas do pgAdmin/DBeaver).



Passo 2

Em cada conexão, execute:



Passo 3

Observe o resultado

```
BEGIN;  
CALL comprar_produto(1);
```

- A primeira execução realiza a compra.
- A segunda recebe a mensagem: *Produto esgotado*.

O que está acontecendo nos bastidores

A primeira transação **bloqueia** o registro com FOR UPDATE.

A segunda transação tenta ler o mesmo produto → **aguarda o desbloqueio**.

Quando o primeiro COMMIT é feito, o estoque = 0.

A segunda transação verifica e **faz ROLLBACK**.

Resultado esperado

Cliente	Ação	Resultado
Cliente 1	Compra iniciada	✓ Sucesso
Cliente 2	Compra simultânea	✗ Cancelada – Produto esgotado

📄💡 Isso mostra na prática o papel do isolamento e dos bloqueios.

Cuidados e Boas Práticas

Dicas de controle de concorrência

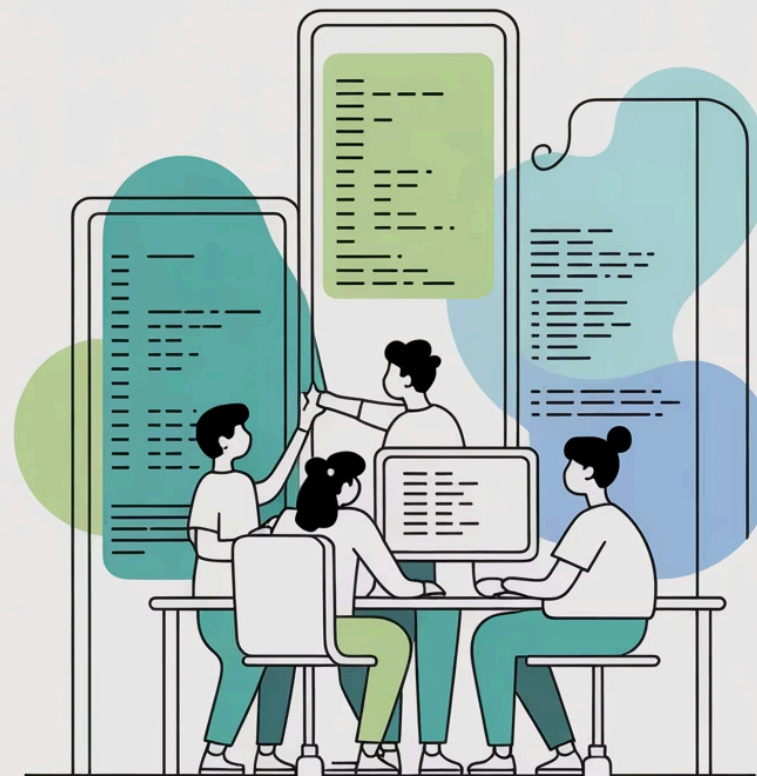
- ❑ Sempre use **FOR UPDATE** em leituras críticas.
- ❑ Evite transações longas (mantêm bloqueios por muito tempo).
- ❑ Capture erros de serialização (SQLSTATE 40001) e repita a operação.
- ❑ Prefira níveis mais altos de isolamento em operações de estoque, pagamentos e finanças.

Exercício Final da Aula

🧩 Desafio: Simule múltiplas compras concorrentes.

1. Use a tabela produto e a procedure comprar_produto.
2. Abra duas conexões e execute a compra ao mesmo tempo.
3. Observe quem consegue o COMMIT e quem sofre ROLLBACK.
4. Explique com suas palavras o papel do FOR UPDATE e do isolamento SERIALIZABLE.

💬 Objetivo: visualizar como o PostgreSQL garante integridade mesmo em situações de concorrência.



✓ Resumo e Encerramento

Concorrência

Concorrência = múltiplas transações simultâneas.

Controle

O banco usa **bloqueios** e **níveis de isolamento** para manter a integridade.

Níveis

READ COMMITTED é rápido, **SERIALIZABLE** é seguro.

FOR UPDATE

O FOR UPDATE é a principal ferramenta para evitar problemas de estoque e inconsistência.